# 15

# 机制：地址转换

在实现 CPU 虚拟化（virtualization）时，我们重点讨论了一种通用机制：**受限直接执行**（limited direct execution，**LDE**）。LDE 的思路很简单：大部分时候让程序直接在硬件上运行；但在某些关键时刻（比如进程（process）发起系统调用（system call），或者发生时钟中断（timer interrupt）时），要设法让操作系统介入，确保事情按“正确”的方式进行。这样，操作系统只需一点硬件支持，就能尽量不去打扰正在运行的程序，实现高效的虚拟化；同时，它又在这些关键时刻**介入**（interposing），从而保持对硬件的控制。高效和控制，是所有现代操作系统的两个主要目标。

虚拟化内存时，我们会采用类似的策略：既要提供所需的虚拟化，又要兼顾高效和控制。要高效，就得借助硬件支持。这种支持起初很简陋（比如只有几个寄存器），但会逐渐变得相当复杂（比如 TLB、页表（page table）支持等，后面你会看到）。要控制，操作系统就得确保任何应用程序都只能访问自己的内存；为了让应用程序之间互不干扰、让操作系统不受应用程序破坏，这里同样需要硬件帮忙。最后，我们还希望虚拟内存（virtual memory）系统在*灵活性*上多做一点：程序应该能按自己想要的任何方式使用地址空间（address space），从而让系统更便于编程。由此，我们得到了更具体的关键问题：

关键问题：
如何高效、灵活地虚拟化内存
如何构建高效的内存虚拟化？如何提供应用程序需要的灵活性？如何控制应用程序能访问哪些内存位置，确保它的内存访问受到恰当的限制？如何高效地做到这一切？

我们要用的通用技术可以看作对受限直接执行（limited direct execution）这一通用方法的补充，叫作**基于硬件的地址转换**（hardware-based address translation），简称**地址转换**（address translation）。有了地址转换，硬件会转换每一次内存访问（例如取指令、加载或存储），把指令给出的**虚拟**（virtual）地址换成所需信息实际所在的**物理**（physical）地址。因此，每次内存引用时，硬件都会做一次地址转换，把应用程序的内存引用重定向（redirect）到它们在内存中的实际位置。

当然，光靠硬件还不能虚拟化内存，硬件只是提供了高效实现虚拟化的底层机制。操作系统必须在关键时刻介入，把硬件设置好，让转换正确进行；因此，操作系统必须**管理内存**

（manage memory）：记录哪些位置空闲、哪些在用，并审慎地干预，以保持对内存使用的控制。

和以前一样，这一切工作的目标都是营造一个美丽的**假象**（illusion）：程序拥有自己私有的内存，里面放着它自己的代码和数据。而在这个虚拟的现实背后，是丑陋的物理真相：CPU（一个或多个）在不同程序之间来回切换，许多程序实际上同时共享着内存。通过虚拟化，操作系统（在硬件的帮助下）把丑陋的机器现实变成了一个有用、强大又好用的抽象（abstraction）。

## 15.1 假设

我们对内存虚拟化的最初尝试会非常简单，简单得近乎可笑。笑吧，尽管笑；等你费力去弄懂 TLB、多级页表（multi-level page table）和其他各种技术奇迹的来龙去脉时，很快就轮到操作系统笑你了。不喜欢被操作系统嘲笑？那恐怕由不得你，操作系统就是这个脾气。

具体来说，我们暂时假设用户的地址空间必须连续地放在物理内存（physical memory）中。为了简单，我们还假设地址空间不会太大，确切地说，它小于物理内存的大小。最后，我们还假设每个地址空间的大小完全相同。这些假设听起来不现实也没关系，我们会一步步放宽它们，最终实现符合实际的内存虚拟化。

## 15.2 一个例子

为了更好地理解实现地址转换要做什么、为什么需要这样的机制，我们来看一个简单的例子。假设某个进程的地址空间如图 15.1 所示。我们要分析的是一小段代码：它从内存中加载一个值，加上 3，再把结果存回内存。这段代码用 C 语言写出来，大概是这样的：

```
void func() {
    int x = 3000; // 谢谢你，Perry。
    x = x + 3;    // 我们关注的这行代码
    ...
```

编译器会把这行代码翻译成汇编，大致如下（x86 汇编）。在 Linux 上可以用 `objdump`，在 Mac 上可以用 `otool` 来反汇编（disassemble）：

```
128: movl 0x0(%ebx), %eax  ;把 0+ebx 处的值加载到 eax
132: addl $0x03, %eax      ;eax 寄存器加 3
135: movl %eax, 0x0(%ebx)  ;把 eax 存回内存
```

这段代码比较直观。它假设 `x` 的地址已经存放在寄存器 `ebx` 中，然后用 `movl` 指令（意为“longword” move，即长字传送）把该地址上的值加载到通用寄存器 `eax`。下一条指令把 `eax` 加 3，最后一条指令把 `eax` 中的值存回内存中原来的位置。

从图 15.1（第 3 页）可以看到代码和数据在进程地址空间中的布局：这三条指令位于地址 128（在靠近顶部的代码段中），变量 `x` 的值位于地址 15 KB（在靠近底部的栈（stack）中）。图中栈上 `x` 所在的位置标出了它的初始值 3000。

从进程的角度看，这几条指令运行时会发生以下内存访问：

- 从地址 128 取指令

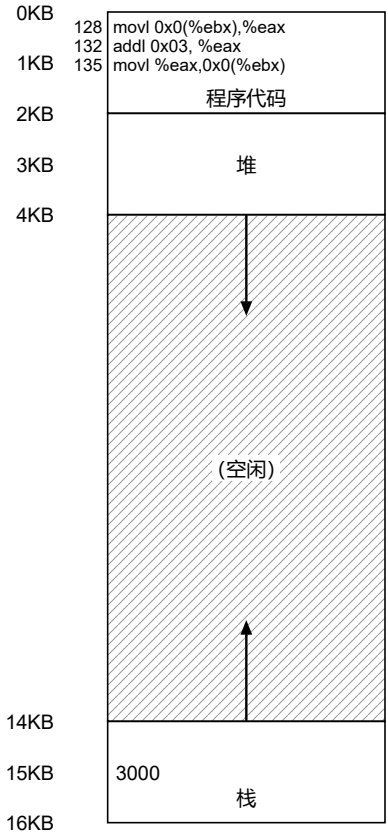



图 15.1：**一个进程及其地址空间**

- 执行这条指令（从地址 15 KB 加载）
- 从地址 132 取指令
- 执行这条指令（不访问内存）
- 从地址 135 取指令
- 执行这条指令（存储到地址 15 KB）

提示：介入很强大

介入（interposition）是一种通用且强大的技术，在计算机系统中常常能收到很好的效果。在虚拟化内存时，硬件会介入每一次内存访问，把进程发出的每个虚拟地址（virtual address）转换成所需信息实际存放的物理地址（physical address）。不过，介入这种技术的适用范围远不止于此；事实上，几乎在任何定义良好的接口上都可以介入，用来增加新功能，或者改进系统的其他方面。这类做法的一个常见好处是**透明**（transparency）：介入通常不需要改动客户端的接口，因此客户端本身也无需任何修改。

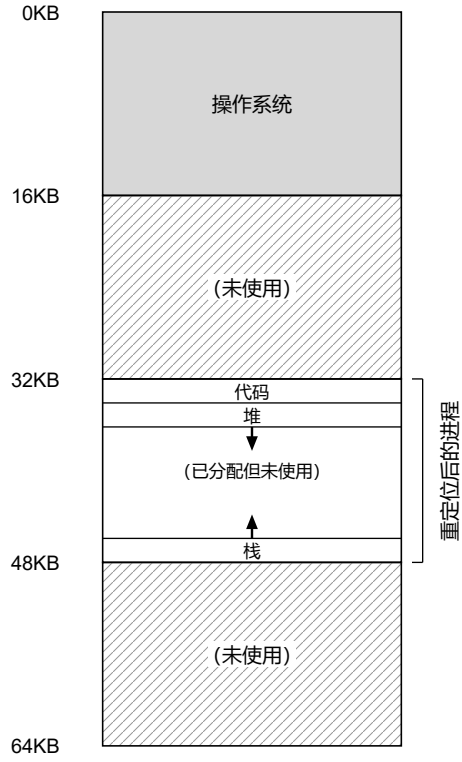



图 15.2：**只有一个重定位进程的物理内存**

从程序的角度看，它的**地址空间**（address space）从地址 0 开始，最大到 16 KB；它产生的所有内存引用都应该落在这个范围内。然而，为了虚拟化内存，操作系统希望把这个进程放到物理内存的其他地方，不一定从地址 0 开始。于是问题来了：怎样才能在内存中**重定位**（relocate）这个进程，同时又对进程**透明**（transparent）？地址空间实际位于另外某个物理地址，怎样才能营造出虚拟地址空间（virtual address space）从 0 开始的假象？

图 15.2 展示了这个进程的地址空间放进内存后，物理内存可能的样子。可以看到，操作系统把物理内存的第一个槽位留给了自己，并把上面例子中的进程重定位到了从物理地址 32 KB 开始的槽位。另外两个槽位（16 KB–32 KB 和 48 KB–64 KB）是空闲的。

## 15.3 动态（基于硬件的）重定位

为了对基于硬件的地址转换（hardware-based address translation）有个初步了解，我们先讨论它最早的形式。20 世纪 50 年代末，最早的一批分时（time sharing）机器引入了一个简单的思想，叫作**基址加界限**（base and bounds）；这种技术也叫**动态重定位**（dynamic relocation），下文两种说法会交替使用 [SS74]。

 © 2008–23, ARPACI-DUSSEAU

补充：基于软件的重定位

早期还没有硬件支持的时候，有些系统纯粹靠软件方法，实现了一种比较粗糙的重定位。其基本技术叫作**静态重定位**（static relocation）：由一个叫**加载器**（loader）的软件拿到即将运行的可执行文件，把其中的地址改写为物理内存中期望的偏移位置。

例如，假设有一条指令从地址 1000 加载数据到寄存器（例如 `movl 1000, %eax`），而程序的地址空间是从地址 3000 开始加载的（而不是程序以为的 0），那么加载器就会改写这条指令，给每个地址都加上偏移量（offset）3000（例如改成 `movl 4000, %eax`）。这样就实现了对进程地址空间的简单静态重定位（static relocation）。

然而，静态重定位（static relocation）有不少问题。首先也是最重要的一点：它不提供保护。进程可能产生错误的地址，从而非法访问其他进程甚至操作系统的内存；一般来说，要实现真正的保护，很可能离不开硬件支持 [WL+93]。另一个缺点是，地址空间一旦放好，之后就很难再重定位到别的位置 [M65]。

具体来说，每个 CPU 内需要两个硬件寄存器：一个叫**基址**（base）寄存器，另一个叫**界限**（bounds）寄存器（有时也叫**限制**（limit）寄存器）。有了这对基址和界限寄存器（base and bounds registers），我们就能把地址空间放到物理内存中任何想放的位置，同时确保进程只能访问它自己的地址空间。

在这种方案下，每个程序在编写和编译时，都假定自己是从地址零开始加载的。但当程序开始运行时，操作系统会决定把它加载到物理内存的什么位置，并把基址寄存器（base register）设为这个地址。在上面的例子中，操作系统决定把进程加载到物理地址 32 KB，于是就把基址寄存器设为这个值。

进程运行起来以后，有意思的事情就开始了。此时，进程产生的任何内存引用，都会被处理器按以下方式**转换**（translated）：

**物理地址 = 虚拟地址 + 基址**

进程产生的每个内存引用都是一个**虚拟地址**（virtual address）；硬件把基址寄存器的内容加到这个地址上，得到的就是可以发给内存系统的**物理地址**（physical address）。

为了更好地理解这一点，我们来跟踪一条指令执行时的情况。具体来说，就看前面那段指令序列中的这一条：

```
128: movl 0x0(%ebx), %eax
```

程序计数器（PC）被设为 128。硬件需要取这条指令时，先把这个值加上基址寄存器的值 32 KB（32768），得到物理地址 32896，然后从这个物理地址取出指令。接着，处理器开始执行这条指令。执行过程中，进程会从虚拟地址 15 KB 加载数据，处理器拿到这个地址后，同样加上基址寄存器的值（32 KB），得到最终的物理地址 47 KB，从而取到所需的内容。

把虚拟地址转换为物理地址，正是我们所说的**地址转换**（address translation）技术：硬件拿到进程以为自己正在引用的虚拟地址，把它转换为数据实际所在的物理地址。由于这种

提示：基于硬件的动态重定位

有了动态重定位（dynamic relocation），一点点硬件就能起到很大作用。具体来说，**基址**（base）寄存器用来把（程序产生的）虚拟地址转换为物理地址，**界限**（bounds）或**限制**（limit）寄存器则确保这些地址不超出地址空间的范围。两者合在一起，就提供了一种简单而高效的内存虚拟化。

地址重定位发生在运行时，而且即使进程已经开始运行，我们也可以移动它的地址空间，所以这种技术通常称为**动态重定位**（dynamic relocation） [M65]。

你现在可能会问：那个界限寄存器（bounds register，也叫限制寄存器）去哪了？这不是基址（base）加界限（bounds）的方法吗？没错，确实是。你可能已经猜到，界限寄存器（bounds register）是用来帮助实现保护的。具体来说，处理器会先检查内存引用是否在界限（*bounds*）之内，以确保它合法；在上面的简单例子中，界限寄存器（bounds register）始终设为 16 KB。如果进程产生的虚拟地址大于（或等于）界限（bounds），或者是负数，CPU 就会触发异常（exception），这个进程多半会被终止。所以，界限（bounds）的作用就是确保进程产生的所有地址都合法，都在进程的“界限”之内——这你大概也猜到了。

需要指出的是，基址和界限寄存器（base and bounds registers）是位于芯片上的硬件结构（每个 CPU 一对）。人们有时把处理器中负责地址转换的这部分称为**内存管理单元**（memory management unit，MMU）；随着我们开发出更复杂的内存管理技术，还会往 MMU 里加入更多电路。

顺便说一下，界限寄存器（bounds register）有两种定义方式。一种（如上所述）是保存地址空间的大小，硬件先用它检查虚拟地址，再加上基址（base）。另一种是保存地址空间末尾的物理地址（physical address），硬件先加上基址（base），再检查地址是否在界限（bounds）之内。两种方式在逻辑上是等价的；为简单起见，我们通常假设采用前一种。

### 地址转换示例

为了更详细地了解基址加界限（base and bounds）的地址转换，我们来看一个例子。假设有一个进程，它的地址空间大小为 4 KB（没错，小得不现实），被加载到了物理地址 16 KB 处。下面是几次地址转换的结果：

| 虚拟地址 | | 物理地址 |
|---|---|---|
| 0 | → | 16 KB |
| 1 KB | → | 17 KB |
| 3000 | → | 19384 |
| 4400 | → | 故障（越界） |

从这个例子可以看出，只要把基址（base）加到虚拟地址上（虚拟地址完全可以看作地址空间内的偏移量（offset）），就能得到相应的物理地址。只有当虚拟地址“太大”或为负数时，才会出现故障（fault），并触发异常。

 © 2008–23, ARPACI-DUSSEAU

## 15.4 硬件支持：总结

现在我们来总结一下需要硬件提供哪些支持（另见第 7 页的图 15.3）。首先，正如 CPU 虚拟化一章讨论过的，我们需要两种不同的 CPU 模式。操作系统运行在**特权模式**（privileged mode，也称**内核模式**，kernel mode）下，可以访问整台机器；应用程序运行在**用户模式**（user mode）下，能做的事情受到限制。CPU 当前处于哪种模式，可以用一个位来表示，这个位也许就存放在某种**处理器状态字**（processor status word）中；在某些特殊情况下（例如系统调用，或其他某种异常或中断（interrupt）），CPU 会切换模式。

硬件还必须提供**基址和界限寄存器**（base and bounds registers）本身；因此，每个 CPU 都多了一对寄存器，它们属于 CPU 的**内存管理单元**（MMU）。用户程序运行时，硬件会把基址（base）加到用户程序产生的虚拟地址上，以此转换每个地址。硬件还必须能检查地址是否有效，这要靠界限寄存器（bounds register）和 CPU 内部的一些电路来完成。

硬件还应该提供专门的指令来修改基址和界限寄存器（base and bounds registers），让操作系统在运行不同进程时修改它们。这些指令属于**特权**（privileged）指令，只有在内核（或特权）模式（kernel/privileged mode）下才能修改这些寄存器。试想一下，如果用户进程能在运行时随意修改基址寄存器，它能造成多大的破坏[1]。想想看！然后赶紧把这种阴暗的念头从脑子里赶出去，噩梦就是这种可怕的东西做成的。

最后，当用户程序试图非法访问内存（使用“越界”地址）时，CPU 必须能够产生**异常**（exception）。这时，CPU 应该停止执行用户程序，转而让操作系统的“越界”**异常处理程序**（exception handler）运行。操作系统的处理程序随后决定如何应对，在这种情况下多半会终止该进程。同样，如果用户程序试图修改（特权的）基址和界限寄存器（base and bounds

补充：数据结构——空闲链表

操作系统必须记录物理内存中哪些部分没有被使用，这样才能给进程分配内存。能完成这项任务的数据结构（data structure）当然有很多种；最简单的一种（我们这里就假设用它）是**空闲链表**（free list），它就是一个链表（linked list），记录物理内存中当前没有被使用的各段范围。

| **硬件要求** | **说明** |
|---|---|
| 特权模式 | 防止用户模式的进程执行特权操作 |
| 基址/界限寄存器 | 每个 *CPU* 需要一对寄存器，用于支持地址转换和界限检查 |
| 能够转换虚拟地址并检查是否在界限内 | 负责转换和检查界限的电路；这里的实现非常简单 |
| 用于更新基址/界限的特权指令 | 操作系统必须能在用户程序运行前设置这些值 |
| 用于注册异常处理程序的特权指令 | 操作系统必须能告诉硬件发生异常时运行哪段代码 |
| 能够触发异常 | 进程试图执行特权指令或访问越界内存时 |

图 15.3：**动态重定位：硬件要求**

[1] wreak 后面除了接 havoc（破坏），还能接别的词吗？[W17]

 © 2008–23, ARPACI-DUSSEAU

registers），CPU 也应该触发异常，并运行“试图在用户模式下执行特权操作（privileged operation）”的处理程序。CPU 还必须提供某种方法，让操作系统告诉它这些处理程序在哪里，因此还需要再加几条特权指令（privileged instruction）。

## 15.5 操作系统层面的问题

硬件为支持动态重定位（dynamic relocation）提供了新功能，操作系统也随之面临一些必须处理的新问题；硬件支持和操作系统管理结合起来，就实现了一个简单的虚拟内存。具体来说，要实现我们这个基址加界限（base and bounds）版本的虚拟内存，操作系统必须在几个关键时刻介入。

第一，创建进程时，操作系统必须采取行动，在内存中为进程的地址空间找到位置。好在根据我们的假设，每个地址空间都（a）小于物理内存，并且（b）大小相同，所以这件事对操作系统来说很容易：它可以把物理内存看成一个槽位数组，记录每个槽位是空闲还是在用。创建新进程时，操作系统要搜索一个数据结构（通常称为**空闲链表**（free list）），为新的地址空间找到位置，然后把这个位置标记为已用。如果地址空间大小可变，情况就会复杂得多，不过这个问题我们留到后面的章节再讨论。

我们来看一个例子。从图 15.2（第 4 页）可以看到，操作系统把物理内存的第一个槽位留给了自己，并把上面例子中的进程重定位到了从物理地址 32 KB 开始的槽位。另外两个槽位（16 KB–32 KB 和 48 KB–64 KB）是空闲的，因此空闲链表中应该有这两项。

第二，进程终止时（无论是正常退出，还是因行为不端被强行杀掉），操作系统必须做一些工作，回收它的全部内存，供其他进程或操作系统使用。进程终止后，操作系统会把它的内存放回空闲链表，并视需要清理相关的数据结构。

第三，发生上下文切换（context switch）时，操作系统还必须多做几步。毕竟，每个 CPU 上只有一对基址和界限寄存器（base and bounds registers），而每个程序被加载到内存中不同的物理地址，所以每个运行中的程序（running program）对应的寄存器值都不一样。因此，操作系统在进程之间切换时，必须保存和恢复基址和界限寄存器（base and bounds registers）。具体来说，当操作系统决定停止运行某个进程时，必须把基址和界限寄存器（base and bounds registers）的值保存到内存中，放在每个进程各有一份的某种结构里，比如**进程结构**（process structure）或**进程控制块**（process control block，PCB）。同

| **操作系统要求** | **说明** |
|---|---|
| 内存管理 | 需要为新进程分配内存；<br>回收已终止进程的内存；<br>一般通过**空闲链表**管理内存 |
| 基址/界限管理 | 必须在上下文切换时正确设置基址/界限 |
| 异常处理 | 发生异常时要运行的代码；<br>通常会终止出错的进程 |

图 15.4：**动态重定位：操作系统的职责**

 © 2008–23, ARPACI-DUSSEAU

| **操作系统 @ 启动（内核模式）** | **硬件** | **（还没有程序）** |
|---|---|---|
| **初始化陷阱表** | | |
| | 记住以下处理程序的地址……<br>系统调用处理程序<br>定时器处理程序<br>非法内存访问处理程序<br>非法指令处理程序 | |
| **启动中断定时器** | | |
| | 启动定时器；X 毫秒后中断 | |
| **初始化进程表** | | |
| **初始化空闲链表** | | |

图 15.5：**受限直接执行（动态重定位）@ 启动**

样，当操作系统恢复运行某个进程（或第一次运行它）时，必须把 CPU 上的基址和界限寄存器（base and bounds registers）设为这个进程对应的值。

需要指出的是，当进程停止（即没有在运行）时，操作系统可以相当容易地把它的地址空间从内存中的一个位置移到另一个位置。具体做法是：操作系统先暂停调度这个进程；然后把地址空间从当前位置复制到新位置；最后更新保存在进程结构（process structure）中的基址寄存器，让它指向新位置。等进程恢复运行时，它的（新）基址寄存器被恢复，进程又开始运行，完全不知道自己的指令和数据已经搬到了内存中一个全新的位置。

第四，操作系统必须提供**异常处理程序**（exception handler），也就是前面讨论过的那些要被调用的函数；操作系统在启动时（通过特权指令）安装这些处理程序。例如，如果某个进程试图访问界限（bounds）之外的内存，CPU 就会触发异常，操作系统必须做好准备，在这种异常发生时采取行动。操作系统通常的反应是毫不留情：它多半会终止这个出错的进程。操作系统应当严密保护它所运行的机器，因此，哪个进程要是试图访问不该访问的内存、执行不该执行的指令，操作系统绝不会对它客气。再见了，不守规矩的进程，很高兴认识你。

图 15.5 和图 15.6（第 10 页）用时间线的形式展示了硬件和操作系统之间的大部分交互。第一张图展示操作系统在启动时为让机器就绪做了什么；第二张图展示的是进程 A 开始运行后发生的事情，请注意，它的内存转换完全由硬件处理，操作系统没有介入。某个时刻（第二张图的中间），发生了时钟中断，操作系统切换到进程 B，而 B 执行了一次“错误的加载”（访问了一个非法的内存地址）；这时操作系统必须介入，终止该进程并做好清理：释放 B 的内存，并从进程表（process table）中删除它的条目。从图中可以看出，我们仍然遵循**受限直接执行**（limited direct execution）的基本思路。大多数情况下，操作系统只需把硬件设置好，然后让进程直接在 CPU 上运行；只有在进程行为不端时，操作系统才需要介入。

## 15.6 小结

本章借助虚拟内存中的一种特定机制，也就是**地址转换**（address translation），扩展了受限直接执行（limited direct execution）的概念。有了地址转换，操作系统就能控制进程

 © 2008–23, ARPACI-DUSSEAU

| **操作系统 @ 运行（内核模式）** | **硬件** | **程序（用户模式）** |
|---|---|---|
| **启动进程 A：** | | |
| 在进程表中分配条目 | | |
| 为进程分配内存 | | |
| 设置基址/界限寄存器 | | |
| **从陷阱返回**（进入 A） | | |
| | 恢复 A 的寄存器 | |
| | 切换到**用户模式** | |
| | 跳转到 A 的（初始）PC | |
| | | **进程 A 运行** |
| | | 取指令 |
| | 转换虚拟地址 | |
| | 执行取指 | |
| | | 执行指令 |
| | 如果是显式的加载/存储： | |
| | 确保地址合法 | |
| | 转换虚拟地址 | |
| | 执行加载/存储 | |
| | | （A 运行……） |
| | **时钟中断** | |
| | 切换到**内核模式** | |
| | 跳转到处理程序 | |
| **处理定时器** | | |
| 决定：停止 A，运行 B | | |
| 调用 `switch()` 例程 | | |
| 保存 A 的寄存器 | | |
| 到 A 的进程结构 | | |
| （包括基址/界限） | | |
| 恢复 B 的寄存器 | | |
| 从 B 的进程结构 | | |
| （包括基址/界限） | | |
| **从陷阱返回**（进入 B） | | |
| | 恢复 B 的寄存器 | |
| | 切换到**用户模式** | |
| | 跳转到 B 的 PC | |
| | | **进程 B 运行** |
| | | 执行错误的加载 |
| | 加载越界； | |
| | 切换到**内核模式** | |
| | 跳转到陷阱处理程序 | |
| **处理陷阱** | | |
| 决定终止进程 B | | |
| 释放 B 的内存 | | |
| 释放 B 在进程表中的条目 | | |

图 15.6：**受限直接执行（动态重定位）@ 运行时**

 © 2008–23, ARPACI-DUSSEAU

的每一次内存访问，确保访问不超出地址空间的界限（bounds）。这项技术之所以高效，关键在于硬件支持：硬件为每次访问快速完成转换，把虚拟地址变成物理地址。这一切对被重定位（relocated）的进程都是透明的：进程完全不知道自己的内存引用正在被转换，于是形成了一个美妙的假象。

我们还介绍了一种具体的虚拟化方式，叫作基址加界限（base and bounds），也叫动态重定位（dynamic relocation）。基址加界限（base and bounds）的虚拟化相当高效，因为只需多一点硬件逻辑，把基址寄存器加到虚拟地址上，再检查进程产生的地址是否在界限（bounds）之内就行了。基址加界限（base and bounds）还提供了保护（protection）：操作系统和硬件联手，确保任何进程都无法产生超出自身地址空间的内存引用。保护（protection）无疑是操作系统最重要的目标之一；没有保护，操作系统就无法控制机器（如果进程可以随意覆盖内存，它们很容易干出覆盖陷阱表（trap table）、进而接管整个系统这类坏事）。

遗憾的是，这种简单的动态重定位（dynamic relocation）技术确实存在效率问题。例如，从图 15.2（第 4 页）可以看到，被重定位（relocated）的进程占用了从 32 KB 到 48 KB 的物理内存；但由于进程的栈和堆（heap）都不大，两者之间的空间全都白白浪费了。这种浪费通常称为**内部碎片**（internal fragmentation），因为已分配单元内部的空间没有被充分利用（即形成了碎片），造成了浪费。按我们目前的做法，即使物理内存足够容纳更多进程，我们也只能把地址空间放进固定大小的槽位里，所以可能产生内部碎片[2]。因此，我们需要更精巧的机制，以便更好地利用物理内存、避免内部碎片。我们首先要尝试的，是对基址加界限（base and bounds）稍作推广，也就是**分段**（segmentation），下一章就来讨论它。

[2]另一种办法是在地址空间里放一个固定大小的栈，紧挨在代码区下方，再在栈下方放一个可以增长的堆。但这样会限制灵活性，让递归和深层嵌套的函数调用变得困难，所以我们希望避免这种做法。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[M65] "On Dynamic Program Relocation" by W.C. McGee. IBM Systems Journal, Volume 4:3, 1965, pages 184–199. *This paper is a nice summary of early work on dynamic relocation, as well as some basics on static relocation.*

[P90] "Relocating loader for MS-DOS .EXE executable files" by Kenneth D. A. Pillay. Microprocessors & Microsystems archive, Volume 14:7 (September 1990). *An example of a relocating loader for MS-DOS. Not the first one, but just a relatively modern example of how such a system works.*

[SS74] "The Protection of Information in Computer Systems" by J. Saltzer and M. Schroeder. CACM, July 1974. *From this paper: "The concepts of base-and-bound register and hardware-interpreted descriptors appeared, apparently independently, between 1957 and 1959 on three projects with diverse goals. At M.I.T., McCarthy suggested the base-and-bound idea as part of the memory protection system necessary to make time-sharing feasible. IBM independently developed the base-and-bound register as a mechanism to permit reliable multiprogramming of the Stretch (7030) computer system. At Burroughs, R. Barton suggested that hardware-interpreted descriptors would provide direct support for the naming scope rules of higher level languages in the B5000 computer system." We found this quote on Mark Smotherman's cool history pages [S04]; see them for more information.*

[S04] "System Call Support" by Mark Smotherman. May 2004. people.cs.clemson.edu/~mark/syscall.html. *A neat history of system call support. Smotherman has also collected some early history on items like interrupts and other fun aspects of computing history. See his web pages for more details.*

[WL+93] "Efficient Software-based Fault Isolation" by Robert Wahbe, Steven Lucco, Thomas E. Anderson, Susan L. Graham. SOSP '93. *A terrific paper about how you can use compiler support to bound memory references from a program, without hardware support. The paper sparked renewed interest in software techniques for isolation of memory references.*

[W17] Answer to footnote: "Is there anything other than havoc that can be wreaked?" by Waciuma Wanjohi. October 2017. *Amazingly, this enterprising reader found the answer via google's Ngram viewing tool (available at the following URL:* http://books.google.com/ngrams*). The answer, thanks to Mr. Wanjohi: "It's only since about 1970 that 'wreak havoc' has been more popular than 'wreak vengeance'. In the 1800s, the word wreak was almost always followed by 'his/their vengeance'." Apparently, when you wreak, you are up to no good, but at least wreakers have some options now.*

## 作业（模拟）

程序 `relocation.py` 可以让你观察：在带有基址和界限寄存器（base and bounds registers）的系统中，地址转换是如何进行的。详情请见 README。

## 问题

1. 分别用种子 1、2、3 运行，算出进程产生的每个虚拟地址是在界限（bounds）内还是越界。如果在界限（bounds）内，算出转换后的地址。
2. 用这组选项运行：`-s 0 -n 10`。界限寄存器（bounds register）`-l` 要设成多少，才能保证产生的所有虚拟地址都在界限（bounds）内?
3. 用这组选项运行：`-s 1 -n 10 -l 100`。基址（base）最大可以设成多少，才能让地址空间仍然整个放得进物理内存?
4. 把上面的一些问题再做一遍，但使用更大的地址空间和物理内存。
5. 随机生成的虚拟地址中，有多大比例是有效的？这个比例随界限寄存器（bounds register）的值如何变化？用不同的随机种子运行，界限（bounds）值从 0 一直取到地址空间的最大值，画出一张图。

 © 2008–23, ARPACI-DUSSEAU